

Stage 2 — Orchestrator Pattern & Multi-Agent Systems

Goal: Understand how multiple agents coordinate under an orchestrator. Implement two orchestration styles — code-driven and LLM-driven — and understand when to use each. Build the "Blog Post Factory" with a real Orchestrator, ResearchAgent, WriterAgent, and ReviewerAgent.

Where You Are

In Stage 1 you built:

- A single agent loop (perceive → decide → tool call → observe → answer).
- A tool registry.
- Clean separation: config, tools, agent loop, prompts.

In Stage 2 you extend that into:

- Multiple specialist agents, each with their own tools and system prompt.
- An Orchestrator that plans, delegates, and collects results.
- Two orchestration styles: **code-driven** (you control flow) and **LLM-driven** (model controls flow).
- Structured outputs — agents return typed JSON, not free text, so the orchestrator can reliably parse and route.

Framework Decision for Stage 2

You are still **not using LangChain or LangGraph yet**. Here's why:

Framework	What it adds	Right time
Raw SDK (Stage 1–2)	Full understanding of primitives	Now
LangGraph	Graph-based state + retries + HITL	Stage 3–4 (after you understand state)
LangChain	Multi-provider chains, retrieval	Stage 3 (after memory concepts)
CrewAI	Role-based crews, fast prototyping	Stage 4+

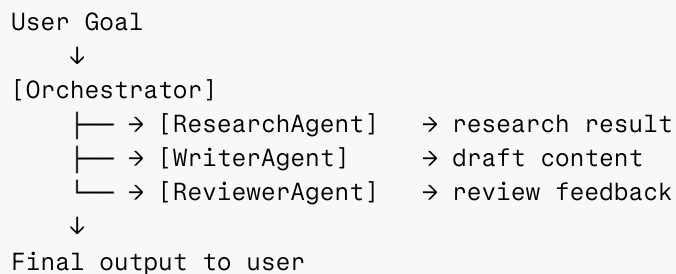
LangGraph is the top-rated production framework in 2026 for complex stateful workflows — it adds graph nodes, edges, checkpoints, and visual debugging. But if you start there before understanding orchestration primitives, you're configuring a graph without knowing what a graph solves. Stage 2 in raw Python will make LangGraph feel obvious when you get to it.

Concept 1 — What is an Orchestrator?

An Orchestrator is an agent (or a code layer) that:

1. Receives a high-level goal from the user.
2. Decomposes it into sub-tasks.
3. Delegates each sub-task to a specialist agent.
4. Collects results.
5. Synthesizes a final output.

The Orchestrator is the **supervisor**. Specialist agents are **workers**. Workers do not talk to each other directly — everything routes through the Orchestrator.



This is the **Supervisor Pattern** — the most common multi-agent orchestration pattern in production systems. The orchestrator has centralized command and control: it receives the user request, decomposes it, delegates to specialists, validates outputs, and synthesizes the final response.

Concept 2 — Two Orchestration Styles

Style A: Code-Driven (Deterministic)

Your code decides which agent runs, in what order.

```
# Code-driven: you define the sequence
research = research_agent.run(topic)
draft    = writer_agent.run(topic, research)
review   = reviewer_agent.run(draft)
final    = synthesize(draft, review)
```

When to use: When the workflow is known in advance. Sequential pipeline, no branching.

Advantage: Predictable, fast, cheap (no extra LLM call for routing).

Disadvantage: Rigid — you hard-code the flow.

Style B: LLM-Driven (Agents-as-Tools)

The Orchestrator is itself an LLM agent. Specialist agents are registered as **tools** in its tool registry. The model decides which agent to call and in what order.

```
# The orchestrator model sees these "tools":
# - run_research_agent(topic)
# - run_writer_agent(topic, research_context)
# - run_reviewer_agent(draft)
# It decides the sequence based on reasoning.
```

When to use: When the workflow needs dynamic routing — the orchestrator might skip a step, repeat a step, or choose different agents based on the task.

Advantage: Flexible, composable, intelligent routing.

Disadvantage: An extra LLM call for orchestration, slightly less predictable.

In Stage 2 you'll build **both** so you understand the trade-off.

Concept 3 — Structured Outputs

When a sub-agent returns free text, the orchestrator must parse it — and LLMs are inconsistent. When a sub-agent returns a typed JSON structure, the orchestrator can always reliably consume it.

OpenAI's Structured Outputs (using Pydantic) guarantees the model will return a valid JSON object matching your schema every time:

```
from pydantic import BaseModel
from openai import OpenAI

class ResearchResult(BaseModel):
    summary: str
    key_points: list[str]
    sources: list[str]

response = client.beta.chat.completions.parse(
    model="gpt-4o-mini",
    messages=[...],
    response_format=ResearchResult,
)
result = response.choices[0].message.parsed # typed ResearchResult object
```

Why this matters for your Creator Orchestrator later:

The PlanningAgent will need to return a structured list of clip proposals. The CopyAgent will return structured asset drafts. If they return free text, the orchestrator must hope it can parse it. If they return typed JSON, the orchestrator always gets exactly what it expects.

Project Structure

```
stage2-orchestrator/
├── .env
├── .env.example
├── requirements.txt
├── config.py
├── tools/
│   ├── __init__.py
│   ├── registry.py
│   ├── search.py           ← simulated web search tool
│   └── calculator.py       ← carried over from Stage 1
├── agents/
│   ├── __init__.py
│   ├── base.py             ← BaseAgent class (shared loop logic)
│   ├── research_agent.py   ← specialist: research a topic
│   ├── writer_agent.py     ← specialist: write content
│   └── reviewer_agent.py   ← specialist: critique content
├── orchestrator/
│   ├── __init__.py
│   ├── schemas.py          ← Pydantic models for structured outputs
│   ├── code_driven.py      ← Style A: deterministic pipeline
│   └── llm_driven.py        ← Style B: orchestrator as agent
└── main.py
```

New pattern: `agents/base.py` introduces a `BaseAgent` class. All specialist agents inherit from it. This is the enterprise pattern — shared agent loop logic lives in one place; specialists only define their system prompt, tools, and output schema.

Step 1 — Setup

requirements.txt

```
openai>=1.30.0
python-dotenv>=1.0.0
pydantic>=2.0.0
```

.env.example

```
OPENAI_API_KEY=your_key_here
ORCHESTRATOR_MODEL=gpt-4o
AGENT_MODEL=gpt-4o-mini
```

Notice two model configs: The Orchestrator (higher reasoning, plans and routes) uses a better model. Specialist agents (focused tasks) use a cheaper model. This is multi-LLM routing at its most basic — you're already doing it.

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
ORCHESTRATOR_MODEL = os.getenv("ORCHESTRATOR_MODEL", "gpt-4o")
AGENT_MODEL = os.getenv("AGENT_MODEL", "gpt-4o-mini")

if not OPENAI_API_KEY:
    raise ValueError("OPENAI_API_KEY is not set in .env")
```

Step 2 — Tools

tools/search.py

```
# Simulated web search – returns fake but realistic results for learning
SCHEMA = {
    "type": "function",
    "function": {
        "name": "web_search",
        "description": (
            "Search the web for information on a topic. "
            "Returns a list of relevant result summaries."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "The search query"
                },
                "max_results": {
                    "type": "integer",
                    "description": "Max number of results to return (1-5)",
                    "default": 3
                }
            },
            "required": ["query"]
        }
    }
}

_FAKE_RESULTS = {
    "ai agents": [
        "AI agents are systems that can perceive their environment, reason about it, "
        "Modern AI agents combine LLMs with tool use, memory, and planning capabilities. "
        "Multi-agent systems coordinate multiple specialized agents for complex tasks."
    ],
    "python programming": [
```

```

        "Python is a high-level, interpreted programming language known for readability.",
        "Python 3.12 introduced significant performance improvements over prior versions.",
        "Python's ecosystem includes over 400,000 packages on PyPI.",
    ],
    "content creation": [
        "Content creators spend an average of 6-8 hours repurposing a single video.",
        "Multi-platform content requires adapting tone, format, and length per channel.",
        "AI tools are reducing content repurposing time by 60-80% for early adopters.",
    ],
}

def run(query: str, max_results: int = 3) -> str:
    """Execute the web search tool."""
    key = query.lower()
    for k, results in _FAKE_RESULTS.items():
        if k in key:
            top = results[:max_results]
            return "\n".join(f"- {r}" for r in top)
    return f"No results found for '{query}'. Try a broader query."

```

tools/registry.py (updated)

```

from tools import search, calculator

REGISTRY = {
    "web_search": (search.SCHEMA, search.run),
    "calculator": (calculator.SCHEMA, calculator.run),
}

def get_schemas(tool_names: list[str] = None) -> list:
    """
    Return tool schemas – optionally filtered by name.
    This is the first taste of per-agent tool access control:
    each agent only gets the tools it needs.
    """
    if tool_names is None:
        return [schema for schema, _ in REGISTRY.values()]
    return [
        REGISTRY[name][0]
        for name in tool_names
        if name in REGISTRY
    ]

def execute(tool_name: str, arguments: dict) -> str:
    if tool_name not in REGISTRY:
        return f"Error: unknown tool '{tool_name}'"
    _, run_fn = REGISTRY[tool_name]
    try:
        return run_fn(**arguments)
    except Exception as e:
        return f"Tool execution error: {e}"

```

Key addition: `get_schemas(tool_names=[...])` lets each agent get only its own tools. The `ResearchAgent` only gets `web_search`. The `WriterAgent` gets no tools (just LLM reasoning). This

is your first implementation of **per-agent tool scoping** — a core enterprise guardrail concept.

Step 3 — BaseAgent

This is the most architecturally important file in Stage 2.

agents/base.py

```
import json
from openai import OpenAI
from config import OPENAI_API_KEY
from tools.registry import get_schemas, execute

class BaseAgent:
    """
    Base class for all specialist agents.
    Contains the shared agent loop from Stage 1.
    Subclasses define:
        - name: agent identifier
        - system_prompt: behavior and role
        - allowed_tools: which tools this agent can use
        - model: which LLM model to use
    """

    name: str = "BaseAgent"
    system_prompt: str = "You are a helpful assistant."
    allowed_tools: list[str] = [] # empty = no tools
    model: str = "gpt-4o-mini"
    max_iterations: int = 6

    def __init__(self):
        self.client = OpenAI(api_key=OPENAI_API_KEY)

    def run(self, user_message: str, context: str = "") -> str:
        """
        Run this agent on a user message.
        Optional context is appended to the system prompt.
        Returns the agent's final text response.
        """
        system = self.system_prompt
        if context:
            system += f"\n\n--- CONTEXT PROVIDED ---\n{context}"

        messages = [
            {"role": "system", "content": system},
            {"role": "user", "content": user_message},
        ]

        tools = get_schemas(self.allowed_tools) if self.allowed_tools else []
        call_kwargs = {"model": self.model, "messages": messages}
        if tools:
            call_kwargs["tools"] = tools
            call_kwargs["tool_choice"] = "auto"
```

```

for iteration in range(self.max_iterations):
    response = self.client.chat.completions.create(**call_kwargs)
    message = response.choices[0].message

    if message.tool_calls:
        # Execute tools and feed results back
        messages.append(message)
        for tool_call in message.tool_calls:
            tool_name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)
            result = execute(tool_name, arguments)
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": result,
            })
        call_kwargs["messages"] = messages
    else:
        return message.content or ""

return "Error: agent loop exceeded max iterations."

```

What BaseAgent gives you:

- The full Stage 1 loop is now reusable across all specialist agents.
- Each specialist only configures 4 things: name, prompt, allowed_tools, model.
- Adding a new specialist agent in your Creator Orchestrator later is just: create a class, set 4 attributes, done.

Step 4 — Specialist Agents

agents/research_agent.py

```

from agents.base import BaseAgent
from config import AGENT_MODEL

class ResearchAgent(BaseAgent):
    name = "ResearchAgent"
    model = AGENT_MODEL
    allowed_tools = ["web_search"]
    system_prompt = """You are a specialist research agent.

```

Your job is to research a given topic thoroughly and return:

1. A concise summary (2-3 sentences).
2. 3-5 key points about the topic.
3. Any important facts or statistics.

Always use the web_search tool to gather information before summarizing.
Do not make up information – only use what the search returns.

Be factual, concise, and structured in your response.
"""

agents/writer_agent.py

```
from agents.base import BaseAgent
from config import AGENT_MODEL

class WriterAgent(BaseAgent):
    name = "WriterAgent"
    model = AGENT_MODEL
    allowed_tools = [] # No tools – pure LLM writing
    system_prompt = """You are a specialist content writing agent.

Your job is to write a well-structured, engaging short blog post (200-300 words).

When given a topic and research context:
1. Write a compelling title.
2. Write 2-3 paragraphs of engaging content.
3. End with a clear takeaway or call to action.

Write in clear, accessible language. No jargon. No filler.
Use the research context provided to ensure accuracy.
"""
```

agents/reviewer_agent.py

```
from agents.base import BaseAgent
from config import AGENT_MODEL

class ReviewerAgent(BaseAgent):
    name = "ReviewerAgent"
    model = AGENT_MODEL
    allowed_tools = [] # No tools – pure LLM reasoning
    system_prompt = """You are a specialist content review agent.

Your job is to critically review a blog post draft and provide structured feedback.

For every draft you receive:
1. Rate overall quality: APPROVE, REVISE, or REJECT.
2. List 2-3 specific strengths.
3. List 2-3 specific improvements needed (if any).
4. Suggest one concrete rewrite for the weakest sentence.

Be direct and specific. Vague feedback is not useful.
If the draft is strong, say so clearly – don't invent problems.
"""
```

Step 5 — Structured Output Schemas

orchestrator/schemas.py

These are the typed contracts between agents and the orchestrator. The orchestrator always gets predictable, typed data back.

```
from pydantic import BaseModel

class ResearchOutput(BaseModel):
    """Structured output from ResearchAgent."""
    summary: str
    key_points: list[str]
    facts: list[str]

class WritingOutput(BaseModel):
    """Structured output from WriterAgent."""
    title: str
    content: str
    word_count: int

class ReviewOutput(BaseModel):
    """Structured output from ReviewerAgent."""
    verdict: str # "APPROVE", "REVISE", or "REJECT"
    strengths: list[str]
    improvements: list[str]
    suggested_rewrite: str

class CampaignPlan(BaseModel):
    """
    Future use: structured output for Creator Orchestrator.
    Shown here to preview where Stage 2 leads.
    """
    topic: str
    target_platforms: list[str]
    proposed_clips: list[str]
    suggested_posts: list[str]
```

Step 6 — Style A: Code-Driven Orchestrator

This is the deterministic pipeline. You control the order.

orchestrator/code_driven.py

```
from openai import OpenAI
from config import OPENAI_API_KEY, AGENT_MODEL
from agents.research_agent import ResearchAgent
from agents.writer_agent import WriterAgent
from agents.reviewer_agent import ReviewerAgent
from orchestrator.schemas import ResearchOutput, WritingOutput, ReviewOutput

class CodeDrivenOrchestrator:
    """
    Deterministic pipeline orchestrator.
    Flow: research → write → review → synthesize.
    The orchestrator controls exactly which agent runs and when.
    """

    def __init__(self):
        self.research_agent = ResearchAgent()
        self.writer_agent = WriterAgent()
        self.reviewer_agent = ReviewerAgent()
        self.client = OpenAI(api_key=OPENAI_API_KEY)

    def _parse_structured(self, agent, user_msg: str, schema, context: str = ""):
        """
        Run an agent and parse its output into a structured Pydantic model.
        Uses OpenAI structured outputs for guaranteed JSON.
        """
        system = agent.system_prompt
        if context:
            system += f"\n\n--- CONTEXT ---\n{context}"

        system += (
            "\n\nIMPORTANT: You MUST respond with valid JSON only, "
            "matching the exact schema provided. No other text."
        )

        response = self.client.beta.chat.completions.parse(
            model=AGENT_MODEL,
            messages=[
                {"role": "system", "content": system},
                {"role": "user", "content": user_msg},
            ],
            response_format=schema,
        )
        return response.choices[0].message.parsed

    def run(self, topic: str) -> dict:
        """
        Run the full blog post pipeline for a given topic.
        Returns a dict with research, draft, and review.
        """
        print(f"\n{'='*60}")
        print(f"CODE-DRIVEN ORCHESTRATOR: '{topic}'")
        print(f"{'='*60}")
```

```

# — Step 1: Research —————
print("\n[Step 1] ResearchAgent running...")
raw_research = self.research_agent.run(f"Research this topic: {topic}")

# Parse into structured output
research: ResearchOutput = self._parse_structured(
    self.research_agent,
    f"Research this topic and return structured output: {topic}",
    ResearchOutput
)
print(f"  Summary: {research.summary[:80]}...")
print(f"  Key points: {len(research.key_points)} found")

# — Step 2: Write —————
print("\n[Step 2] WriterAgent running...")
research_context = (
    f"Summary: {research.summary}\n"
    f"Key Points:\n" +
    "\n".join(f"- {p}" for p in research.key_points) +
    f"\nFacts:\n" +
    "\n".join(f"- {f}" for f in research.facts)
)

draft: WritingOutput = self._parse_structured(
    self.writer_agent,
    f"Write a blog post about: {topic}",
    WritingOutput,
    context=research_context
)
print(f"  Title: {draft.title}")
print(f"  Word count: {draft.word_count}")

# — Step 3: Review —————
print("\n[Step 3] ReviewerAgent running...")
draft_context = f"TITLE: {draft.title}\n\nCONTENT:\n{draft.content}"

review: ReviewOutput = self._parse_structured(
    self.reviewer_agent,
    "Review this blog post draft:",
    ReviewOutput,
    context=draft_context
)
print(f"  Verdict: {review.verdict}")
print(f"  Strengths: {len(review.strengths)}")
print(f"  Improvements: {len(review.improvements)}")

# — Step 4: Synthesize —————
print("\n[Step 4] Synthesizing final output...")

return {
    "topic":      topic,
    "research":   research.model_dump(),
    "draft":      draft.model_dump(),
    "review":     review.model_dump(),
    "pipeline":   "code-driven",
}

```

Step 7 — Style B: LLM-Driven Orchestrator (Agents-as-Tools)

Now specialist agents become **tools** that the orchestrator model can call dynamically.

orchestrator/llm_driven.py

```
import json
from openai import OpenAI
from config import OPENAI_API_KEY, ORCHESTRATOR_MODEL
from agents.research_agent import ResearchAgent
from agents.writer_agent import WriterAgent
from agents.reviewer_agent import ReviewerAgent

class LLMDrivenOrchestrator:
    """
    Dynamic orchestrator where specialist agents are tools.
    The LLM decides which agent to call, when, and in what order.
    """

    def __init__(self):
        self.client = OpenAI(api_key=OPENAI_API_KEY)
        self.research_agent = ResearchAgent()
        self.writer_agent = WriterAgent()
        self.reviewer_agent = ReviewerAgent()
        self._context_store = {} # holds inter-agent results

    # — Agent wrappers as tool functions —————
    def _call_research_agent(self, topic: str) -> str:
        result = self.research_agent.run(f"Research: {topic}")
        self._context_store["research"] = result
        return result

    def _call_writer_agent(self, topic: str) -> str:
        context = self._context_store.get("research", "")
        result = self.writer_agent.run(
            f"Write a blog post about: {topic}",
            context=context
        )
        self._context_store["draft"] = result
        return result

    def _call_reviewer_agent(self, draft: str = "") -> str:
        if not draft:
            draft = self._context_store.get("draft", "No draft found.")
        return self.reviewer_agent.run("Review this draft:", context=draft)

    # — Tool schemas for the orchestrator —————
    AGENT_TOOL_SCHEMAS = [
        {
            "type": "function",
            "function": {
                "name": "call_research_agent",
                "description": "Calls the ResearchAgent to research a topic. Returns
```

```

        "parameters": {
            "type": "object",
            "properties": {
                "topic": {"type": "string", "description": "Topic to research"},
            },
            "required": ["topic"]
        }
    },
    {
        "type": "function",
        "function": {
            "name": "call_writer_agent",
            "description": "Calls the WriterAgent to write a blog post. Should be used after research is complete.",
            "parameters": {
                "type": "object",
                "properties": {
                    "topic": {"type": "string", "description": "Topic to write about"},
                },
                "required": ["topic"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "call_reviewer_agent",
            "description": "Calls the ReviewerAgent to review the most recent draft. Should be used after a draft is written.",
            "parameters": {
                "type": "object",
                "properties": {
                    "draft": {
                        "type": "string",
                        "description": "Optional: pass a specific draft to review"
                    }
                },
                "required": []
            }
        }
    }
]

```

SYSTEM_PROMPT = """You are an orchestrator agent managing a blog post creation process.

You have access to three specialist agents as tools:

- call_research_agent: researches a topic
- call_writer_agent: writes a blog post (requires research context)
- call_reviewer_agent: reviews the most recent draft

Your job:

1. Research the topic first.
2. Then write a blog post using the research.
3. Then review the draft.
4. Finally, return a concise summary of what was produced.

Always follow this sequence. Do not skip steps.

```
"""
```

```
def run(self, topic: str) -> dict:
    print(f"\n{'='*60}")
    print(f"LLM-DRIVEN ORCHESTRATOR: '{topic}'")
    print(f"{'='*60}")

    messages = [
        {"role": "system", "content": self.SYSTEM_PROMPT},
        {"role": "user", "content": f"Create a blog post about: {topic}"},
    ]

    agent_dispatch = {
        "call_research_agent": self._call_research_agent,
        "call_writer_agent": self._call_writer_agent,
        "call_reviewer_agent": self._call_reviewer_agent,
    }

    for iteration in range(10):
        print(f"\n[Orchestrator iteration {iteration + 1}]")

        response = self.client.chat.completions.create(
            model=ORCHESTRATOR_MODEL,
            messages=messages,
            tools=self.AGENT_TOOL_SCHEMAS,
            tool_choice="auto",
        )

        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)

            for tool_call in message.tool_calls:
                agent_name = tool_call.function.name
                arguments = json.loads(tool_call.function.arguments)
                print(f" → Orchestrator calling: {agent_name}({arguments})")

                fn = agent_dispatch[agent_name]
                result = fn(**arguments)
                print(f" ← Result length: {len(result)} chars")

                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result,
                })
        else:
            # Orchestrator gave final summary
            return {
                "topic": topic,
                "research": self._context_store.get("research", ""),
                "draft": self._context_store.get("draft", ""),
                "summary": message.content,
                "pipeline": "llm-driven",
            }
```

```
return {"error": "Orchestrator loop exceeded max iterations."}
```

Step 8 — Main Entry Point

main.py

```
import json
from orchestrator.code_driven import CodeDrivenOrchestrator
from orchestrator.llm_driven import LLMDrivenOrchestrator

def print_result(result: dict):
    print("\n" + "="*60)
    print("FINAL RESULT")
    print("="*60)
    pipeline = result.get("pipeline", "unknown")
    print(f"Pipeline: {pipeline}\n")

    if pipeline == "code-driven":
        draft = result.get("draft", {})
        review = result.get("review", {})
        print(f"Title: {draft.get('title', '')}")
        print(f"Content:\n{draft.get('content', '')}")
        print(f"\nReview verdict: {review.get('verdict', '')}")
        print(f"Strengths: {review.get('strengths', [])}")
        print(f"Improvements: {review.get('improvements', [])}")
    else:
        print(f"Summary: {result.get('summary', '')}")
        print(f"\nDraft preview:\n{result.get('draft', '')[:300]}...")

def main():
    print("Stage 2 – Orchestrator & Multi-Agent Demo")
    print("="*60)
    print("1. Code-Driven Orchestrator (deterministic pipeline)")
    print("2. LLM-Driven Orchestrator (agents-as-tools)")
    print("3. Both (compare outputs)\n")

    choice = input("Choose (1/2/3): ").strip()
    topic = input("Enter topic (e.g. 'AI agents'): ").strip()
    if not topic:
        topic = "AI agents"

    if choice in ("1", "3"):
        orchestrator = CodeDrivenOrchestrator()
        result = orchestrator.run(topic)
        print_result(result)

    if choice in ("2", "3"):
        orchestrator = LLMDrivenOrchestrator()
        result = orchestrator.run(topic)
        print_result(result)
```

```
if __name__ == "__main__":  
    main()
```

Run It

```
pip install -r requirements.txt  
python main.py
```

Test Scenarios (run all of them)

Test 1 — Code-Driven, known topic

```
Topic: AI agents  
Pipeline: 1 (code-driven)
```

Watch: 3 agents run in sequence. Each prints its iteration. Observe how `_context_store` passes research → writer → reviewer.

Test 2 — LLM-Driven, same topic

```
Topic: AI agents  
Pipeline: 2 (llm-driven)
```

Watch: The orchestrator decides the order. Usually it follows research → write → review, but notice the output — it made that decision itself. Count the orchestrator iterations.

Test 3 — Compare both pipelines

```
Topic: content creation  
Pipeline: 3 (both)
```

Compare outputs. Notice the LLM-driven result often has a richer summary because the orchestrator synthesizes it. The code-driven result is more structured and faster.

Test 4 — Intentionally break the flow

Try injecting a bad topic like "xyzzzyabcdef123" to see how the research agent handles missing results and how the writer handles thin context.

What to Observe and Understand

Q&A to test yourself

Q: What is the fundamental difference between an agent and a tool?

A: An agent has a system prompt, an internal loop, and reasoning capabilities. A tool is just a function. In the LLM-driven orchestrator, agents ARE exposed as tools — but internally they're full agents with their own loops running inside the tool call.

Q: In the LLM-driven orchestrator, what happens if the model calls `call_writer_agent` before `call_research_agent`?

A: The writer runs with empty context (no research). The system prompt tells the orchestrator to research first, but it's not guaranteed — the LLM could decide differently. This is the core trade-off of LLM-driven routing: flexible but not 100% deterministic. Stage 4 guardrails solve this.

Q: Why does `BaseAgent` use `allowed_tools = []` for `WriterAgent` and `ReviewerAgent`?

A: Because they don't need tools — they operate purely on reasoning. Giving them tools they don't need wastes tokens and risks them calling something unexpected. Per-agent tool scoping is a guardrail.

Q: Where in the code is multi-LLM routing already happening?

A: In `config.py`: `ORCHESTRATOR_MODEL = gpt-4o` and `AGENT_MODEL = gpt-4o-mini`. The orchestrator uses a stronger model (better planning). Specialists use a cheaper model (focused tasks). This is exactly the routing pattern the Creator Orchestrator will use.

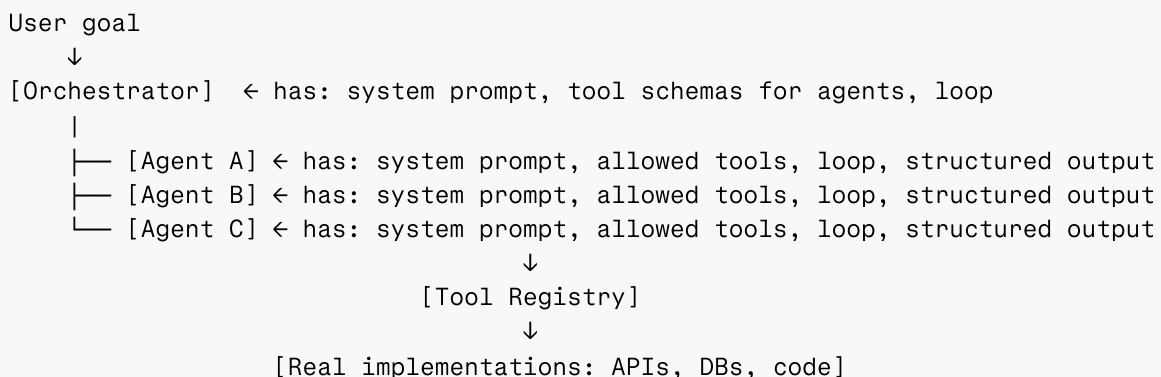
Q: What would you change to add a 4th agent — `SEOAgent` — that adds SEO keywords to the draft after writing?

A: Create `agents/seo_agent.py` (subclass `BaseAgent`, set prompt, no tools). In code-driven: add one line `seo_result = self.seo_agent.run(...)` after writing. In LLM-driven: add one tool schema to `AGENT_TOOL_SCHEMAS`. Zero changes to `BaseAgent`, zero changes to existing agents.

The Architecture Evolution Checkpoint

After Stage 2, your mental model should be:

Every multi-agent system is:



This is the same architecture as Anthropic's multi-agent research system, the same as OpenAI's Agents SDK orchestrator-worker pattern, and the same as your Creator Campaign Orchestrator will use. You've now built it from first principles.

What Stage 3 Adds

Right now, agents have no memory. Every `orchestrator.run(topic)` call starts fresh. Ask the orchestrator "continue where we left off" and it has no idea what you mean.

Stage 3 adds:

- Short-term memory: session-scoped conversation history in Postgres.
- Long-term memory: a vector store (pgvector) where past results are stored and retrieved semantically.
- Episodic memory: run logs so you can replay what each agent did.

The `BaseAgent.run()` method will gain a `session_id` parameter. The structured outputs you built in Stage 2 will be stored in the DB. The agent loop will automatically inject relevant past context on every call.

Everything you built in Stage 2 carries forward unchanged — Stage 3 only adds around it.

Quick Reference: Code-Driven vs LLM-Driven

	Code-Driven	LLM-Driven
Who decides agent order	Your code	The orchestrator LLM
Predictability	100% — always same sequence	~90% — LLM can vary
Speed	Faster (no orchestrator LLM call)	Slightly slower
Flexibility	Low — hard-coded flow	High — can branch, skip, repeat
Cost	Cheaper	Small extra cost for orchestrator call
Best for	Known, linear workflows	Dynamic tasks, conditional routing
Creator Orchestrator use	MVP v1 pipeline	v2 when adding conditional branches

For your Creator Orchestrator MVP: start with code-driven (known sequence: ingest → plan → copy → brand-guard). Upgrade to LLM-driven when you need the orchestrator to decide "does this clip need a revision pass or is it ready?" dynamically.